BEGINNER TO INTERVIEW READY

Databricks, Data Engineering, SQL, Scala & Azure

Mockline Interview Study Guide

Prepared for Soumiya Ayyappan | Beginner-first explanations, code-reading clues, production scenarios and MCQs

How to use: Study one module, type the code yourself, answer the interview questions aloud, then attempt the scenario MCQs without looking at the answers.

Study roadmap

Phase	Topics	Outcome
1. Foundations	Data roles, database, warehouse, lake, lakehouse, on-premises and cloud	Explain the big picture
2. SQL + Regex	Execution order, keys, NULL handling, joins and code-reading clues	Solve interview SQL
3. Azure + Databricks	ADF, ADLS, Fabric, Synapse, workspace, catalog and compute	Describe an end-to-end platform
4. Pipelines + Delta	Medallion, ingestion, modes, idempotency, CDC/CDF and optimization	Design reliable data flows
5. Spark + Scala	Driver/executor, joins, skew, streaming, Scala basics	Read and explain Spark code
6. Production readiness	Testing, security, cost, audit, documentation and domain questions	Handle scenario questions

1. Data career foundations

Data Engineering vs Data Analytics vs Machine Learning

Area	Simple meaning	Typical output	Common tools
Data Engineering	Builds roads and pipelines that move, clean and store data.	Reliable tables and pipelines	ADF, ADLS, Databricks, Spark, SQL
Data Analytics	Uses prepared data to answer business questions.	KPIs, reports and dashboards	SQL, Power BI, Fabric
Machine Learning	Learns patterns from historical data to predict or classify.	Model predictions	Python, MLflow, scikit-learn, Spark ML

Interview answer: A data engineer prepares trustworthy data, an analyst explains what happened, and an ML engineer or data scientist predicts what may happen next.

Database, warehouse, lake, Delta Lake and lakehouse

Term	Layman analogy	Best for	Key idea
Database	Cashier system	Day-to-day transactions	Fast inserts and updates; structured tables
Data warehouse	Organized supermarket	Business reporting and analytics	Curated structured data and SQL
Data lake	Large storage yard	Raw structured, semi-structured and unstructured data	Flexible, low-cost object storage
Delta Lake	Lake with a register and rules	Reliable lake tables	Parquet plus transaction log, ACID, time travel
Lakehouse	Supermarket connected to the storage yard	BI and ML on one governed platform	Lake flexibility plus warehouse reliability

On-premises, cloud and server-to-cloud ingestion

On-premises means servers are physically managed in the company data center. Cloud means rented, scalable services from providers such as Microsoft Azure. A hybrid setup connects both.

```
On-prem SQL Server -> Self-hosted Integration Runtime -> Azure Data Factory -> ADLS Gen2 -> Databricks -> Delta -> Power BI
```

- Use a self-hosted integration runtime or approved network path to reach private on-premises data.
- Prefer incremental loads after the first full load.
- Encrypt data in transit and at rest, use managed identity, log row counts, and reconcile source versus target.

2. SQL from scratch

Logical SQL execution order

```
FROM/JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT
```

SQL is written as SELECT ... FROM ..., but logically the engine first finds the data source. This explains why a SELECT alias usually cannot be used in WHERE in the same query level.

Stage	Question it answers	Example
FROM/JOIN	Where does data come from?	orders JOIN customers
WHERE	Which individual rows remain?	amount > 1000
GROUP BY	How are rows grouped?	GROUP BY customer_id
HAVING	Which groups remain?	HAVING COUNT(*) > 1
SELECT	Which columns/calculations are returned?	SUM(amount)
ORDER BY	How is the result sorted?	ORDER BY revenue DESC

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
WHERE status = 'ACTIVE'
GROUP BY department
HAVING AVG(salary) > 50000
ORDER BY avg_salary DESC;
```

How to identify the SQL operation from wording

Question words	Think of	Pattern
per department / each customer	GROUP BY	GROUP BY department

more than 2 records per ID	GROUP BY + HAVING	HAVING COUNT(*) > 2
latest row per customer	ROW_NUMBER	PARTITION BY id ORDER BY timestamp DESC
no matching customer	LEFT ANTI / LEFT JOIN IS NULL	WHERE c.id IS NULL
replace missing value	COALESCE	COALESCE(amount,0)
combine and remove duplicates	UNION	UNION
combine and keep duplicates	UNION ALL	UNION ALL
pattern in text	REGEXP / RLIKE	email RLIKE pattern

What is regex?

Regex, or regular expression, is a pattern language used to search, validate, split or replace text. It is like a template for the shape of a string.

Symbol	Meaning	Example
^	start of text	^A
$	end of text	com$
.	any one character	c.t
*	zero or more of previous item	ab*
+	one or more	[0-9]+
[0-9]	one digit	[0-9]
\d	digit	\d{10}
{n}	exactly n times	\d{4}

```
-- Spark SQL: simple email-shape check
SELECT email
FROM customers
WHERE email RLIKE '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$';
```

Caution: Regex validates a pattern, not whether the mailbox really exists.

Candidate key in layman terms

A candidate key is any minimal column, or minimal combination of columns, that can uniquely identify one row. From the candidate keys, one is selected as the primary key.

Real-time analogy: An employee can be uniquely identified by Employee_ID or possibly Corporate_Email. Both are candidate keys. If Employee_ID is chosen as the official identifier, it becomes the primary key. A super key may include unnecessary columns, such as Employee_ID + Name, so it is not minimal.

Key	Meaning	Example
Candidate key	Minimal unique choice	employee_id; corporate_email
Primary key	Chosen candidate key	employee_id
Alternate key	Candidate not chosen	corporate_email
Composite key	Multiple columns together uniquely identify	order_id + product_id
Foreign key	Points to another table key	orders.customer_id -> customers.customer_id

3. Domain knowledge and dataset questioning

Domain knowledge means understanding what the data represents in the business, not only its data type. A strong contributor asks questions before coding.

Area	Questions to ask
Meaning	What does one row represent? What is the business grain?
Keys	Which column uniquely identifies a record? Can it

	change?
Dates	What do created, due and completed dates mean? Which timezone?
Status	What are valid statuses? Can a completed item reopen?
Quality	Can values be NULL? What ranges are valid? Are duplicates allowed?
History	Do we need current state only, or every change?
SLA	Is completed_on <= due_date considered on time? What about weekends?
Ownership	Who owns the source, approves rules and resolves defects?

```
CASE
  WHEN completed_date IS NULL AND current_date() > due_date THEN 'OVERDUE'
  WHEN completed_date <= due_date THEN 'ON_TIME'
  ELSE 'LATE'
END AS completion_status
```

Contributor mindset: Try to diagnose with evidence, documentation and a small reproducible test. Ask for help with a specific finding, not with no investigation.

4. Azure, Fabric and Databricks platform

Core Azure data services

Service	Purpose	Analogy
Azure Data Factory	Ingestion and orchestration	Traffic controller
ADLS Gen2	Cloud data-lake storage	Large governed warehouse
Azure Databricks	Spark-based engineering, analytics and AI	Processing factory
Azure Synapse Analytics	Analytics warehouse and integrated analytics	Reporting warehouse
Microsoft Fabric	SaaS analytics platform with OneLake, engineering, warehouse and Power BI	All-in-one analytics campus
Power BI	Semantic models, reports and dashboards	Business display board
Microsoft Purview	Catalog, classification, lineage and governance	Data librarian

Databricks portal overview

Area	Used for	Real-life analogy
Workspace	Notebooks, files and collaboration	Office desk
Catalog	Discover governed data and AI assets	Library catalog
Compute	Clusters and serverless resources	Engine room
Workflows / Jobs	Schedule and orchestrate tasks	Conveyor-belt controller
SQL	SQL warehouses, editor, queries and dashboards	Analysis counter
Machine Learning	Experiments, models and MLflow	Laboratory
Marketplace	Discover shared data products	App/data store

Catalog, schema, table and metadata

```
Metastore -> Catalog -> Schema -> Table/View/Volume/Function
```

A metastore is the top governance container. A catalog groups data by domain or environment. A schema groups related objects. A table stores rows and columns. Metadata is data about data, such as names, columns, owners, location, format and permissions.

```
main.sales.orders
```

Here main is the catalog, sales is the schema and orders is the table.

Managed identity and access connector

A managed identity is an Azure-managed identity for a resource, so code does not need embedded passwords or storage keys. An Azure Databricks Access Connector can provide a managed identity used with governed storage access. Grant only required permissions and avoid secrets in notebooks.

5. Ingestion, pipelines and medallion architecture

What is data ingestion?

Data ingestion is bringing data from a source into the platform. Sources can be databases, APIs, files, message streams or SaaS systems. Batch ingestion moves data at intervals; streaming ingestion processes arriving events continuously.

Medallion layers

Layer	Purpose	Typical rules
Bronze	Keep raw source data with audit columns	Minimal changes; preserve evidence
Silver	Clean, standardize, deduplicate and conform	Schema, NULL defaults, quality rules
Gold	Business-ready aggregates and data products	KPIs, dimensions, facts and reports

Some projects call these raw, staging, curated or presentation layers. Names vary, but the responsibilities should be clear.

Pipeline design and documentation process

```
Source -> Ingestion -> Bronze -> Validation -> Silver -> Business rules -> Gold -> Consumers
```

- Document source and target, owner, schedule, volume, schema, keys, watermark, rules, reject handling and SLA.
- Add source_system, source_file, ingestion_time, batch_id, record_hash and processing_status where appropriate.
- Record row counts, rejected rows, start/end time, run ID and error message in an audit table.
- Test in development, perform unit and integration tests, obtain review, deploy through CI/CD, then monitor production.

NULL defaults during transformation

```
SELECT COALESCE(discount, 0) AS discount,
       COALESCE(country, 'UNKNOWN') AS country
FROM bronze_orders;
```

Use a default only when the business confirms its meaning. NULL may mean unknown, not applicable or missing. Replacing every NULL with 0 can create incorrect business results.

Write modes: overwrite, append, ignore and error

Mode	First run	Second run	Use carefully when
append	Adds rows	Adds again, possibly duplicates	Every incoming row is genuinely new
overwrite	Creates/replaces target	Replaces existing content	Full refresh is intentional
ignore	Creates target	Does nothing if target exists	Create-once flows
error/errorifexists	Creates target	Fails if target exists	Accidental replacement must be prevented

saveAsTable normally uses error-if-exists unless another mode is specified. overwriteSchema permits replacing the table schema during overwrite; it does not itself make incremental loads unique.

6. Idempotency, audit and recovery

Idempotency means rerunning the same input produces the same correct final state, without duplicate business records.

```
MERGE INTO silver.orders t
USING staged_orders s
ON t.order_id = s.order_id
WHEN MATCHED AND s.updated_at >= t.updated_at THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

- Use a stable business key or event ID.
- Deduplicate the source before MERGE.
- Store batch ID and processed-file information.
- Advance checkpoints/watermarks only after successful processing.
- Reconcile source, inserted, updated, deleted, rejected and target counts.

Audit field	Purpose
run_id / batch_id	Identify one execution or source batch
source_system	Know origin
source_file or event_id	Trace exact input
record_count	Reconcile volumes
start_time / end_time	Measure duration and SLA
status / error	Troubleshoot outcome
created_by / updated_by	Accountability

7. Delta Lake, CDC and CDF

Delta Lake essentials

- Delta table = Parquet data files plus _delta_log transaction history.
- ACID gives reliable commits. Time travel reads retained earlier versions.
- MERGE supports upsert logic. Delta does not automatically enforce a business primary key.

CDC versus CDF

Term	Meaning	Direction
CDC	Change Data Capture: capture inserts, updates and deletes from a source system	Source -> downstream
CDF	Delta Change Data Feed: expose row-level changes made to a Delta table	Delta table -> consumer

The uniqueness comes from your stable key, event ID, sequence, deduplication and MERGE condition, not from the word CDC or CDF.

```
-- Enable CDF on a Delta table
ALTER TABLE silver.customers SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

```
# Read changes from a retained starting version
spark.read.format('delta')\
  .option('readChangeFeed','true')\
  .option('startingVersion', 10)\
  .table('main.silver.customers')
```

Delta optimization features

Feature	Purpose	Exam/interview clue
Partitioning	Create directory groups by low/moderate-cardinality column	Large table commonly filtered by date
Partition pruning	Skip partitions not matching filter	WHERE order_date = ...
OPTIMIZE	Compact many small files	Small-file problem
Z-ORDER	Co-locate related column values for skipping	Frequent selective filters; legacy/manual layout tuning
Liquid clustering	Flexible clustering that can evolve	Avoid rigid partition choices for large tables
VACUUM	Remove old unreferenced files after retention	Can affect time travel if too aggressive

8. Spark internals and performance

Driver, executors, transformations and actions

The driver builds the plan and coordinates the application. Executors run tasks on partitions. Transformations such as select and filter are lazy. Actions such as count, collect and write trigger execution.

Catalyst Optimizer and Photon engine

Catalyst is Spark SQL's optimizer. It analyzes and optimizes logical plans, then chooses a physical plan. Photon is Databricks' vectorized native execution engine for supported SQL and DataFrame workloads. Catalyst decides the plan; Photon can execute supported parts efficiently.

Broadcast, shuffle hash and sort-merge joins

Situation	Likely strategy	Why / risk
One side small	Broadcast hash join	Copy small table to executors and avoid shuffling the large side; ensure it fits memory
Both sides large; suitable build side	Shuffle hash join	Shuffle by key and build hash maps; memory-sensitive
Both sides large	Sort-merge join	Shuffle and sort both sides; scalable for large equi-joins

```
from pyspark.sql.functions import broadcast
result = fact.join(broadcast(dim), 'product_id')
result.explain('formatted')
```

Data skew

Data skew means some partition keys have far more rows than others. One long-running task while others finish quickly is a classic clue.

- Inspect key-frequency distribution and Spark UI task sizes.
- Use Adaptive Query Execution and skew-join handling where appropriate.
- Broadcast the genuinely small side, pre-aggregate, filter early, or salt hot keys when justified.
- Do not blindly increase partitions without identifying the hot key.

Memory, parsing and schema issues

Symptom	Likely cause	First checks
Driver out of memory	collect/toPandas or too much driver state	Keep processing distributed; inspect driver logs
Executor out of memory	Large partitions, join build side, cache pressure	Partition sizes, plan, spilling and memory
Malformed records	Wrong delimiter, quotes, encoding or corrupt input	Sample raw bytes/rows and parser options
Schema mismatch	Wrong inferred type or schema drift	Compare expected vs actual schema; use explicit schema
Unexpected NULLs	Parsing failure or missing source values	Quarantine bad rows; do not silently default

9. Scala basics for Spark interviews

val, var and lazy val

Keyword	Meaning	Can value change?	Can type change?
val	Immutable reference	No	No
var	Mutable variable	Yes	No
lazy val	Evaluated once when first accessed	No after evaluation	No

Correction: var does not mean static. It means the variable can be reassigned. A val reference cannot be reassigned, although a mutable object referenced by val may have its contents changed.

```
val n = 1
// n = 2            // error: reassignment to val
var count = 1
count = 2           // valid
val a = new Array[Int](10)
// a = new Array[Int](12) // error: val reference cannot be reassigned
a(0) = 99           // valid: Array contents are mutable
```

Any and if expressions

```
val x = if (condition) 1 else 0          // Int
val y = if (condition) "positive" else 0 // inferred common type such as Any
```

Scala if is an expression and returns a value. Mixed branch types force a wider common type, often Any, which reduces type safety. Prefer consistent branch types.

String interpolation

```
val name = "Soumiya"
val score = 90
println(s"$name scored $score")
println(f"Average = ${89.456}%.2f")
```

object and singleton object

A Scala object defines exactly one singleton instance for that object declaration. It is useful for shared utilities, entry points and companion objects. A normal class can create many instances with new.

```
object Calculator {
  def add(a: Int, b: Int): Int = a + b
}
println(Calculator.add(2,3))
```

Higher-order function and lambda

A higher-order function takes a function as an argument or returns a function. A lambda is an anonymous function. It is not defined as merely producing different outputs for different inputs.

```
def calculate(a: Int, b: Int, operation: (Int, Int) => Int): Int =
  operation(a, b)

val add = (x: Int, y: Int) => x + y
println(calculate(10, 5, add))       // 15
println(calculate(10, 5, (x,y) => x-y)) // 5
```

Collections and spark-shell

```
cd $SPARK_HOME
./bin/spark-shell

val nums = List(1,2,3,4)
nums.filter(_ % 2 == 0).map(_ * 10)
```

In many interactive Scala shells, entering val n = 1 and then val n = 2 as a new REPL command may be accepted because the REPL creates a new binding. Inside one normal compiled scope, redeclaring the same val name is generally an error.

10. Streaming: DStreams vs Structured Streaming

Area	DStreams	Structured Streaming
Abstraction	Sequence of RDD micro-batches	Unbounded table using DataFrame/Dataset API
State/recovery	Checkpointing required for recovery/state	Checkpoint location stores progress and state metadata
Storage	Processing abstraction, not durable business storage	Also needs an explicit sink for durable output
Recommendation	Legacy API	Preferred modern Spark streaming API

Correction: Structured Streaming checkpoints are recovery metadata, not the business output itself. Write results to Delta, Kafka, files or another durable sink.

11. Development, security, cost and production

Development lifecycle

```
Requirement -> Design -> Develop -> Unit test -> Code review/PR -> QA/SIT -> UAT -> Deploy ->
Monitor -> Support
```

- Write meaningful comments for business rules, not comments that repeat obvious syntax.
- Parameterize environment-specific paths and secrets.
- Use Git branches and pull requests. A feature branch isolates work before review and merge.
- Create unit tests for transformations and integration tests for source-to-target behavior.

Security

- Use Unity Catalog and least privilege.
- Use managed identities/service principals rather than personal credentials.
- Keep secrets in approved secret management, not notebooks.
- Use private networking, encryption, audit logs, masking and row/column policies as required.

Cost

- Use job compute or serverless where suitable; terminate idle interactive compute.
- Filter early, select necessary columns, avoid unnecessary shuffles and repeated full reloads.
- Compact small files and examine query plans.
- Tag resources, monitor DBU/cloud consumption and set budgets/alerts.

Dashboards and alerts in layman terms

- Create a query or semantic model that produces a trusted metric.
- Build a visualization from that result and assemble it into a dashboard.
- Choose a threshold, such as failed_records > 0 or freshness_minutes > 60.
- Create an alert, set evaluation frequency and notification destination, then test it with controlled data.

12. Mock interview: questions and model answers

Tell me about yourself as a beginner data engineer.

I am building skills in SQL, Python, Spark, Databricks and Azure data services. I understand how to ingest data, apply Bronze-Silver-Gold transformations, build idempotent Delta pipelines, validate quality and monitor production. I am a quick learner, but I also verify unfamiliar technology with documentation and small tests.

How do you learn a new technology quickly?

I first understand the business use case and core architecture, follow the official quickstart, build a small end-to-end example, note common errors, and explain it back in simple words. Then I connect it to the project standards for security, cost and deployment.

A pipeline failed after writing some rows. What do you do?

I check the run ID, failed task, logs, target commit history, batch ID and checkpoint. I determine what committed, avoid a blind append, fix the cause, and rerun using idempotent MERGE or processed-batch tracking. Then I reconcile counts.

How would you load data from an on-premises server to Azure?

Use ADF with an approved self-hosted integration runtime or secure network connection, land data in ADLS Bronze, transform in Databricks and publish Delta Silver/Gold tables. Use managed identity, incremental watermarks, audit columns, encryption and reconciliation.

What is data skew?

Uneven key distribution creates very large partitions, so a few Spark tasks run much longer. I inspect key counts and Spark UI, then consider AQE, broadcasting a small side, pre-aggregation, filtering or salting hot keys.

What is idempotency?

Processing the same batch again leaves the correct final state and does not create duplicates. I use stable keys, event IDs, deduplication, MERGE and batch audit tracking.

What is the difference between CDC and CDF?

CDC is the broader pattern of capturing source inserts, updates and deletes. Delta CDF exposes row-level changes made to a Delta table for downstream consumers.

Why is domain knowledge important?

Technical correctness is not enough. I must know the row grain, business keys, status meanings, date rules, SLA and valid values so transformations calculate the intended result.

13. Scenario-based MCQs

1. A query needs departments whose average salary exceeds 50,000. Which filter is correct?

A. WHERE AVG(salary) > 50000

B. HAVING AVG(salary) > 50000

C. ORDER BY AVG(salary)

D. LIMIT 50000

Answer: B - HAVING filters aggregated groups.

2. A rerun appends the same source batch a second time. Best long-term fix?

A. Increase cluster size

B. Use stable batch/key plus deduplication or MERGE

C. Rename the notebook

D. Use collect()

Answer: B - Idempotent writes prevent duplicate business records.

3. A 5 GB fact table joins a 20 MB dimension. What should you evaluate?

A. Broadcast the dimension if it safely fits executor memory

B. Collect both tables

C. Always cartesian join

D. Convert to CSV

Answer: A - Broadcast can avoid shuffling the large table.

4. A support user needs only the last four digits of a phone number.

A. Give the raw table

B. Hide visual column only

C. Apply governed masking/view

D. Email the file

Answer: C - Security should be enforced at the data layer.

5. A Delta table receives duplicate source keys in a MERGE update.

A. MERGE repeatedly

B. Deduplicate to one deterministic row per key

C. Remove ON condition

D. VACUUM

Answer: B - Multiple matches can be ambiguous or fail.

6. Only one Spark task is much slower and processes far more data.

A. Time travel

B. Data skew

C. Masking

D. Lazy val

Answer: B - A hot key or oversized partition creates uneven work.

7. Which Scala declaration permits reassignment?

A. val

B. var

C. object

D. lazy val

Answer: B - var is mutable; its inferred/declared type still remains fixed.

8. Which order is logically correct?

A. SELECT, FROM, WHERE

B. FROM, WHERE, GROUP BY, HAVING, SELECT

C. WHERE, FROM, SELECT

D. ORDER BY, FROM

Answer: B - SQL identifies sources and filters before producing the SELECT result.

14. Final rapid-revision sheet

Trigger phrase	Best concept
before grouping	WHERE
after aggregation	HAVING
unique minimal identifier	Candidate key
search text pattern	Regex
small table in join	Broadcast
one slow huge partition	Data skew
same batch rerun safely	Idempotency
source inserts/updates/deletes	CDC
Delta row-level changes	CDF
replace business-approved NULL	COALESCE
raw/clean/business layers	Bronze/Silver/Gold
Spark plan optimization	Catalyst
native vectorized Databricks execution	Photon
single Scala instance	object
function takes another function	Higher-order function
deferred one-time Scala evaluation	lazy val

Daily preparation routine

- 20 minutes: revise one concept map.
- 30 minutes: type SQL/PySpark/Scala examples.

- 20 minutes: answer five scenario MCQs.
- 15 minutes: explain two topics aloud in simple language.
- 10 minutes: write one production failure and recovery checklist.

Interview rule: Do not guess silently. State your assumption, ask one business question when needed, describe validation, and explain how you prevent data loss, duplicates and unauthorized access.